DarshanEase – System Architecture & Design Documentation

1. Introduction

In recent years, digital platforms have transformed how people access services, including religious and spiritual activities. Many temples receive a large number of devotees every day, which often leads to long queues, overcrowding, and inefficient management of darshan timings. This creates inconvenience for devotees and makes it difficult for temple authorities to manage visitors effectively.

The DarshanEase system is a web-based application developed to simplify and digitize the temple darshan booking process. The platform allows devotees to browse temples, check darshan timings, select available slots, and book darshan tickets online. By enabling advance booking and slot management, the system helps reduce overcrowding and improves the overall experience for devotees.

DarshanEase is developed using modern web technologies such as React.js for the frontend, Node.js with Express.js for the backend, and MongoDB for data storage. The system follows a structured architecture that ensures secure authentication, efficient data management, and a smooth user experience.

Through this platform, temple management can efficiently organize darshan schedules, manage bookings, and monitor slot availability. At the same time, devotees benefit from a convenient and time-saving process to plan their temple visits.

---

2. Problem Statement

Many temples still rely on traditional systems where devotees must stand in long queues to get darshan. This often leads to overcrowding, time wastage, and poor management of temple resources.

The lack of a centralized digital system makes it difficult for devotees to check darshan timings or reserve a slot in advance. As a result, both temple authorities and devotees face challenges in managing the darshan process efficiently.

The DarshanEase system aims to solve this problem by providing an online platform where devotees can easily browse temples, check slot availability, and book darshan tickets in advance.

3. Objectives of the Project

The main objectives of the DarshanEase system are:

- To develop a digital platform for temple darshan booking.
- To reduce overcrowding at temples through slot-based booking.
- To provide devotees with an easy and convenient booking system.
- To allow temple administrators to manage darshan schedules efficiently.
- To provide booking history and confirmation for users.
- To improve the overall experience of temple visits using technology.

4. Scope of the Project

The scope of the DarshanEase project includes designing and developing a web-based platform that enables devotees to book darshan slots online.

The system provides functionalities such as:

- User registration and login
- Temple browsing and information display
- Viewing available darshan slots
- Booking darshan tickets
- Viewing booking history

The system is intended to be used by devotees, temple organizers, and administrators. It can be expanded in the future to include features such as online donations, event management, and mobile application integration.

---

5. Proposed System

The proposed system introduces a digital darshan booking platform that allows users to reserve temple visit slots online.

The system includes three major user roles:

Devotee

Devotees can register, log in, browse temples, view available darshan slots, and book tickets.

Organizer

Temple organizers can manage temple details, create darshan slots, and monitor bookings.

Admin

The administrator has full control over the system, including managing users, temples, and bookings.

The system ensures secure access through authentication and provides real-time information about slot availability.

---

6. Methodology

The DarshanEase system is developed using a full-stack development approach based on the MERN stack.

Frontend Development

The user interface is built using React.js, which provides a responsive and interactive experience for users.

Backend Development

The backend is implemented using Node.js and Express.js, which handle API requests, business logic, and server operations.

Database Management

The system uses MongoDB, a NoSQL database, to store user information, temple details, darshan slots, and booking records.

Authentication

User authentication is implemented using JWT (JSON Web Tokens) and password encryption using bcrypt, ensuring secure login and data protection.

---

1. System Technical Architecture

DarshanEase is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js).

The architecture separates the system into multiple layers including frontend, backend services,

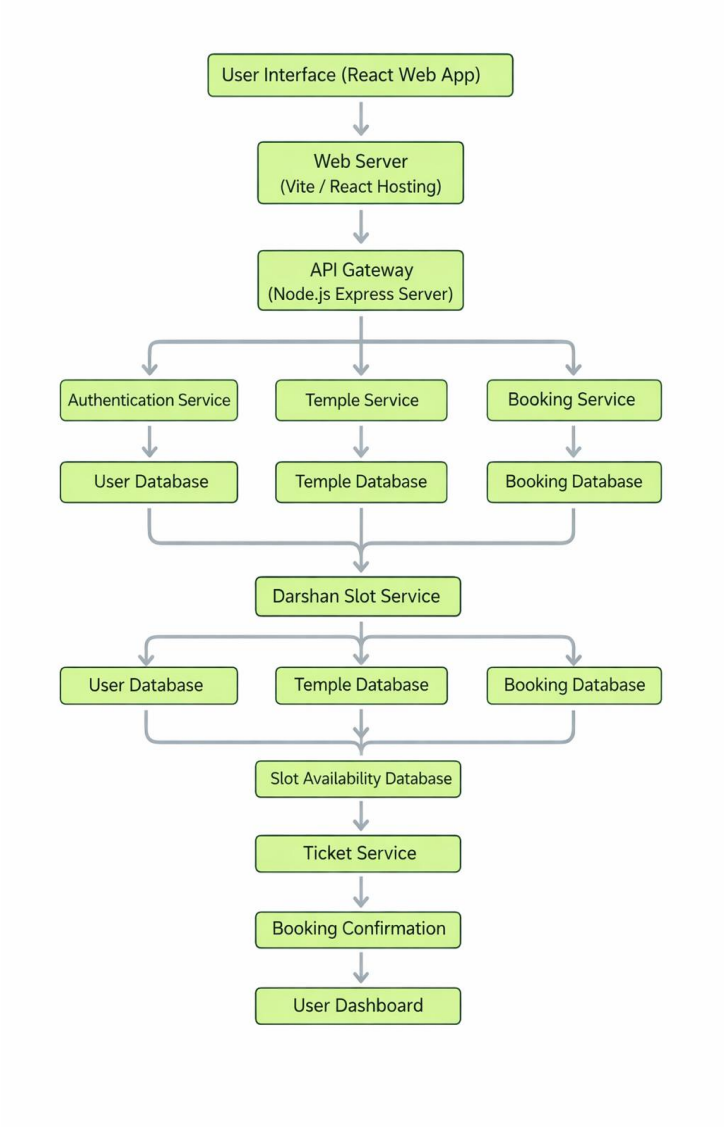
and database management. This layered architecture improves scalability, maintainability, and security.

The React frontend provides the user interface for devotees to browse temples and book darshan slots.

The Node.js + Express backend handles API requests, authentication, and business logic. MongoDB stores

all persistent data such as users, temples, darshan slots, and bookings.

1.1 Architecture Diagram



The architecture of the DarshanEase system follows a multi-tier web architecture designed to ensure scalability, security, and efficient data management. The system is built using modern web technologies including React.js for the frontend, Node.js with Express.js for the backend, and MongoDB for the database. Each layer of the architecture has a specific responsibility that helps in organizing the system effectively.

1. User Interface (Frontend Layer)

The user interface is developed using React.js, which provides a responsive and interactive web environment. This layer allows devotees to browse temples, check available darshan slots, and book tickets. It also enables users to manage their profiles and view their booking history. The frontend communicates with the backend through HTTP requests using APIs.

2. Web Server Layer

The web server hosts the frontend application and delivers the web pages to users through their browsers. It manages client requests and ensures that the user interface loads correctly. This layer acts as the bridge between the user's browser and the backend services.

3. API Gateway / Backend Server

The backend server is developed using Node.js and Express.js, which act as the API gateway for the system. It receives requests from the frontend, processes them, and routes them to the appropriate services such as authentication, temple management, or booking services. The backend handles business logic and ensures secure communication between the frontend and the database.

4. Authentication Service

The authentication service is responsible for managing user registration, login, and secure access to the system. It uses JWT (JSON Web Token) for authentication and bcrypt encryption for password security. This ensures that only authorized users can access protected features such as booking tickets or managing temple data.

5. Temple Management Service

This service manages all information related to temples, including temple details, location, and darshan timings. It allows administrators or organizers to add, update, or manage temple data within the system.

6. Darshan Slot Management Service

The slot management service controls the scheduling and availability of darshan slots. It ensures that devotees can view available time slots and select the most convenient option for booking.

7. Booking and Ticket Service

The booking service handles the entire booking process, including seat reservation, ticket generation, and booking confirmation. Once a user confirms a booking, the system generates a digital ticket and stores the booking details in the database.

8. Database Layer

The database layer is implemented using **MongoDB**, a NoSQL database that stores all system data such as user profiles, temple details, darshan slots, and booking records. **Mongoose** is used as an Object Data Modeling (ODM) tool to interact with the database efficiently.

9. Data Flow Between Components

When a user performs an action such as booking a darshan slot, the request flows from the **React frontend to the backend API server**. The backend validates the request, interacts with the database to retrieve or update data, and sends the response back to the frontend. The frontend then displays the result to the user.

2. Technology Stack

Frontend: React.js, HTML, CSS, JavaScript

Backend: Node.js, Express.js

Database: MongoDB with Mongoose

Authentication: JWT, bcrypt

API Testing: Postman

Version Control: Git & GitHub

3. Data Flow Description

1. The user interacts with the React frontend.

2. The frontend sends HTTP requests using Axios.

3. The Node.js API server receives and processes the request.

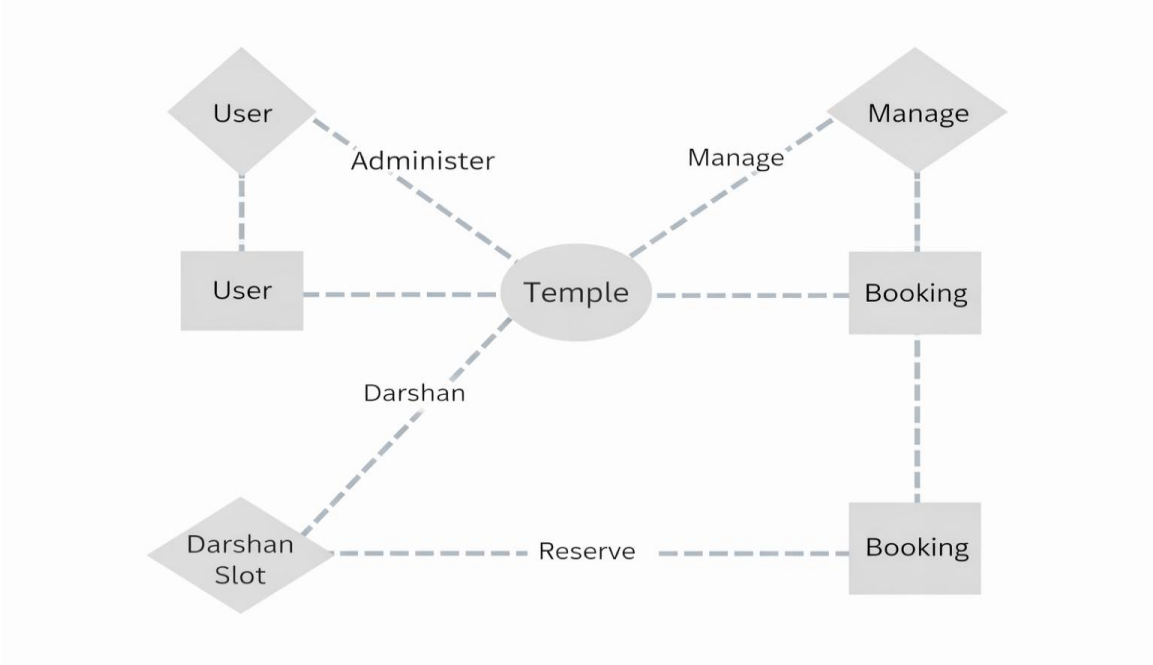
4. Authentication middleware verifies the JWT token.

5. Controllers handle the business logic.

6. Mongoose models interact with the MongoDB database.

7. Data is returned to the frontend and displayed to the user.

4. Entity Relationship (ER) Diagram



USER

- user_id (PK)

- name

- email

- password

- phone

- role

TEMPLE

- temple_id (PK)
- temple_name
- location
- darshan_start_time
- darshan_end_time

DARSHAN_SLOT

- slot_id (PK)
- temple_id (FK)
- date
- start_time
- end_time
- available_seats
- price

BOOKING

- booking_id (PK)
- user_id (FK)
- slot_id (FK)
- booking_date
- total_amount
- status

5. System Features

Devotee Features:

- User registration and login
- Browse temples and darshan slots
- Book darshan tickets
- View booking history
- Cancel bookings
- Provide feedback

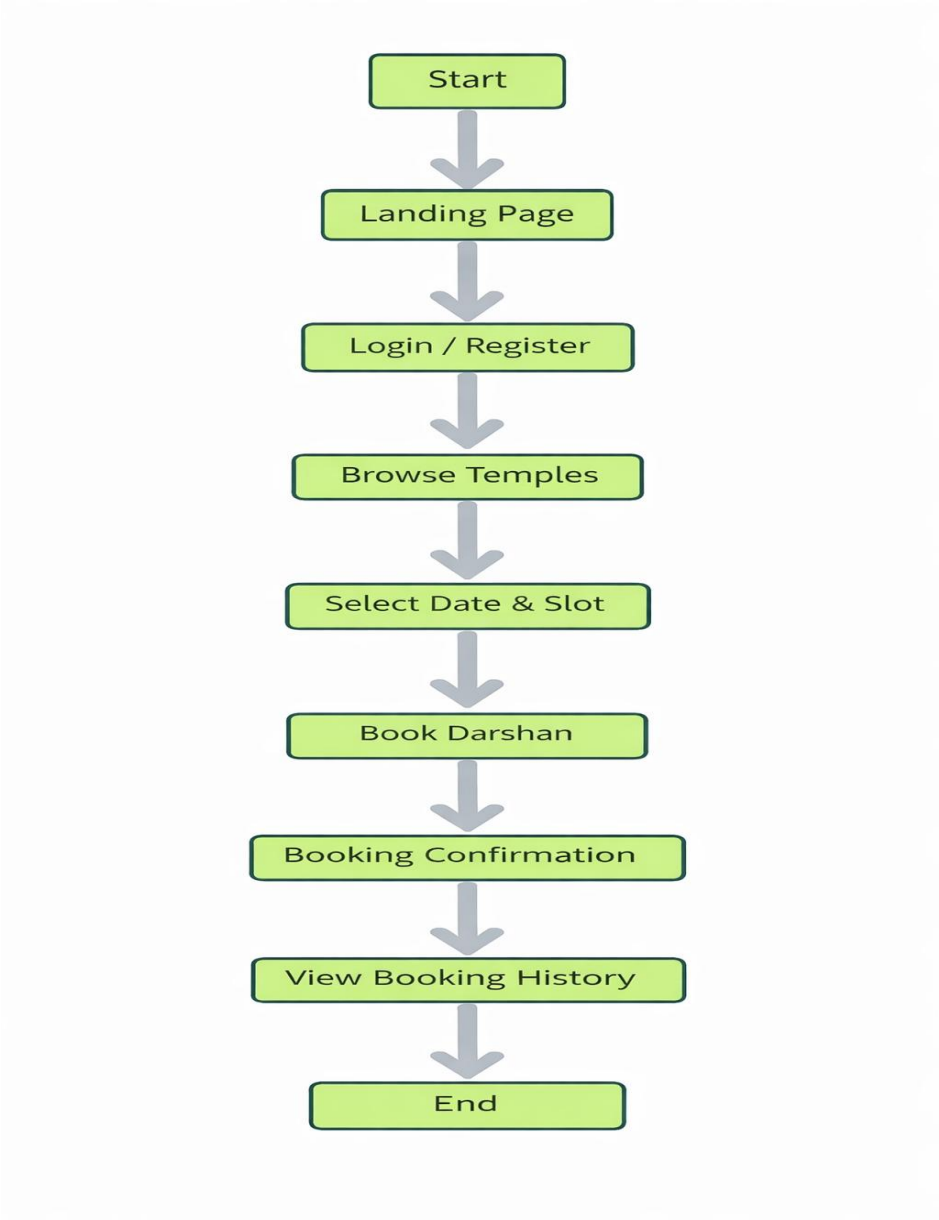
Organizer Features:

- Manage temple information
- Create darshan slots
- Monitor bookings
- Manage seat availability

Admin Features:

- Manage users and organizers
- Monitor temple listings
- Generate reports and analytics

6. User Flow Diagram



User Flow Diagram Description

The **User Flow Diagram** represents the step-by-step process that a devotee follows while using the **DarshanEase platform** to book a darshan slot at a temple. It illustrates how a user interacts with the system from the beginning of the application until the completion of the booking process.

1. Start

The process begins when a user accesses the DarshanEase web application through a web browser. This marks the entry point into the system.

2. Landing Page / Home Page

After opening the application, the user is directed to the home page. The home page provides an overview of the platform and displays information about available temples, features, and navigation options.

3. Login or Registration

If the user is new, they must register by providing basic details such as name, email, phone number, and password. Existing users can log in using their registered credentials. Authentication ensures secure access to the system.

4. Access Profile

After successful login, the user can access their profile section where they can view or update personal information. The profile also stores booking history and account details.

5. Browse Temples

The user can explore a list of temples available in the system. Each temple displays relevant information such as temple name, location, and darshan timings.

6. Temple Selection

The user selects a temple they wish to visit. After selecting a temple, the system displays the available darshan slots for that specific temple.

7. Slot Selection

The user chooses a suitable darshan slot based on available date and time options. The system also shows the number of available seats for each slot.

8. Ticket Booking

Once a slot is selected, the user confirms the booking by specifying the number of devotees and submitting the booking request. The system processes the request and reserves the selected slot.

9. Booking Confirmation

After successful booking, the system generates a booking confirmation that includes booking details such as temple name, slot timing, and ticket information.

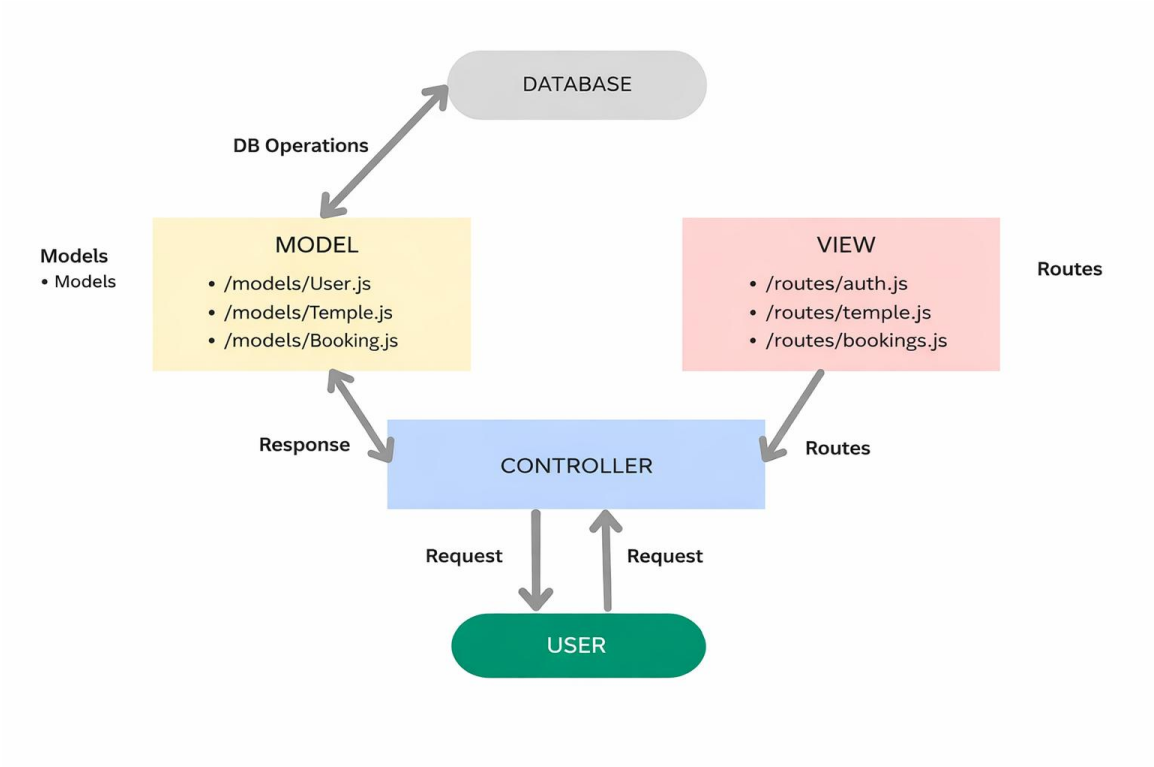
10. View Bookings

The user can access their booking history through the dashboard to view past or upcoming bookings. Users may also cancel bookings if the system allows cancellation.

11. End

The process ends after the user successfully completes the booking and views their booking details.

7. MVC Architecture



MVC Architecture Diagram Description

The **DarshanEase system** follows the **Model–View–Controller (MVC)** architectural pattern. MVC is a widely used software design pattern that separates an application into three main components: **Model, View, and Controller**. This separation helps in organizing the code, improving maintainability, and making the system easier to scale and manage.

In the DarshanEase application, the MVC architecture allows the frontend, backend logic, and database interactions to work independently while maintaining efficient communication between them.

1. Model Layer

The **Model** layer is responsible for handling the application's data and business rules. It defines the structure of the data stored in the database and manages all database operations such as creating, retrieving, updating, and deleting records.

In the DarshanEase system, the model layer is implemented using **MongoDB with Mongoose schemas**. It stores and manages important data such as:

- User information (name, email, password, role)
- Temple details (temple name, location, darshan timings)
- Darshan slot information (date, time, available seats)
- Booking records (user ID, slot ID, booking status)

The model layer interacts directly with the database to store and retrieve the required data for the system.

2. View Layer

The **View** layer represents the user interface of the system. It is responsible for displaying data to the user and collecting input from them. In web applications, the view layer usually consists of frontend components and routes that present information in a structured format.

In the DarshanEase system, the view layer is developed using **React.js**. It provides an interactive interface where users can:

- Browse temples
- View available darshan slots
- Book tickets
- Manage their profile
- View booking history

The view layer communicates with the controller through API requests and displays the responses received from the backend.

3. Controller Layer

The **Controller** acts as the central component that connects the Model and View layers. It receives requests from the user interface, processes them using appropriate logic, and interacts with the model layer to retrieve or update data.

In DarshanEase, the controller layer is implemented using **Node.js and Express.js**. It handles various system operations such as:

- User authentication (login and registration)
- Fetching temple details
- Managing darshan slot availability
- Processing booking requests

- Returning responses to the frontend

The controller ensures that the correct data is retrieved from the database and sent back to the user interface.

4. Database Interaction

The **MongoDB database** stores all application data including users, temples, darshan slots, and booking information. The model layer communicates with the database to perform data operations, while the controller manages the logic behind these interactions.

5. Request and Response Flow

The workflow of the MVC architecture in DarshanEase can be summarized as follows:

1. The **user** interacts with the application through the **React frontend (View)**.
2. The frontend sends an API request to the **Controller**.
3. The controller processes the request and communicates with the **Model**.
4. The model interacts with the **MongoDB database** to retrieve or update data.
5. The controller receives the data from the model.
6. The controller sends a response back to the **View**.
7. The view displays the updated information to the user.

Advantages of MVC in DarshanEase

- Improves code organization and readability
- Allows independent development of frontend and backend
- Makes the system easier to maintain and extend
- Enhances scalability for future features
- Simplifies debugging and testing

8. Project Setup and Configuration

This section explains the step-by-step process used to set up the development environment for the **DarshanEase – Temple Darshan Booking Platform**. The project follows a **full-stack architecture** using **React.js for the frontend, Node.js with Express.js for the backend, and MongoDB as the database**.

8.1 Creating the Project Folder

Description

1. Create a new main project folder and name it **DarshanEase**.
2. Inside this folder, create two subfolders.
3. Name the first folder **Client** (for frontend development).
4. Name the second folder **Server** (for backend development).
5. Open the main **DarshanEase** folder using **Visual Studio Code**.

Project Structure:

```
DarshanEase
│
├── client
│
└── server
```

This structure helps maintain separation between frontend and backend components.

8.2 Client Setup (Frontend – React)

Description

The frontend of the DarshanEase application is developed using **React.js with Vite**.

Steps

Open the **Client folder** in the VS Code terminal.

Run the following command to create the React project:

npm create vite@latest . -- --template react

Select **React** as the framework.

Select a framework:
React

Select **JavaScript** as the variant.

Select a variant:
JavaScript

Navigate to the client folder:

cd client

Install all required dependencies:

npm install

Start the React development server:

npm run dev

The frontend application will run on:

http://localhost:5173

8.3 Server Setup (Backend – Express)

Description

The backend of the DarshanEase system is developed using **Node.js and Express.js**.

Open the **Server folder** in the VS Code terminal.

Initialize the Node.js project:

npm init -y

Create the following files and folders inside the server directory.

Files:

server.js

Folders:

models
controllers
routes
middleware
config

Project Structure:

```
server
│
├── controllers
├── models
├── routes
├── middleware
├── config
└── server.js
```

8.4 Backend Development

Setup Express Server

Create the main server file **server.js**.

Install required packages:

npm install express mongoose cors dotenv body-parser nodemon bcrypt jsonwebtoken

Configure the Express server and include middleware such as:

- **CORS** for cross-origin requests
- **Body Parser** for handling request data
- **dotenv** for environment variables

8.5 User Authentication

User authentication is implemented to provide secure access to the system.

The authentication module includes:

- User registration
- User login
- Password encryption using **bcrypt**
- JWT token generation for secure sessions

Protected routes ensure that only authenticated users can access certain features such as booking tickets.

8.6 Define API Routes

Separate route files are created for different functionalities of the system.

Example routes include:

/api/auth
/api/users
/api/temples
/api/slots
/api/bookings

Each route is handled by Express controllers which perform the required operations and communicate with the database.

8.7 Implement Data Models

The database schema is implemented using **Mongoose models**.

Models created for the DarshanEase system include:

- **User Model** – stores user information and roles
- **Temple Model** – stores temple details
- **Darshan Slot Model** – manages slot scheduling
- **Booking Model** – stores booking records

CRUD operations (Create, Read, Update, Delete) are implemented for each model.

8.8 Error Handling

Error handling middleware is implemented in the backend to ensure smooth system operation.

The system returns appropriate responses including:

- HTTP status codes
- Error messages
- Validation messages

This helps in debugging and maintaining system stability.

8.9 Database Development

Configure MongoDB

The DarshanEase system uses **MongoDB** as the database and **Mongoose** for schema modeling.

Install Mongoose:

npm install mongoose

8.10 Create Database Connection

A database connection is established before performing any backend operations.

Example MongoDB connection configuration:

```
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
```

The connection string is stored in the **.env** file for security.

8.11 Create Schemas and Models

Schemas define how the data is structured in the MongoDB database.

The schemas are created based on the **ER Diagram** of the project.

Example entities:

- Users
- Temples
- Darshan Slots
- Bookings
- Donations

Each schema is converted into a Mongoose model to perform database operations.

8.12 Frontend Development

Setup React Application

The frontend application includes the following steps:

- Create React project using Vite
- Configure **React Router** for navigation
- Install required libraries such as Axios

Design UI Components

User interface components are designed to improve usability.

Main components include:

- Navbar
- Temple listing page
- Darshan slot selection page
- Booking confirmation page
- User dashboard

Styling is implemented using **CSS or Bootstrap**.

Implement Frontend Logic

Frontend functionality includes:

- Integration with backend APIs
- Fetching temple and slot data
- Submitting booking requests
- Displaying booking confirmation

Axios is used for API communication between frontend and backend.

8.13 Project Execution

Step 1 – Run Frontend Application

Open terminal and navigate to the client folder.

cd client

Start the React development server:

npm run dev

The application will run at:

http://localhost:5173

Step 2 – Run Backend Server

Open a new terminal window.

Navigate to the server folder:

cd server

Step 3 – Configure Environment Variables

Inside the **server folder**, create a file named **.env**

Add the MongoDB connection string:

```
MONGO_URI=mongodb://localhost:27017/darshanease
PORT=8000
JWT_SECRET=your_secret_key
```

Step 4 – Start Backend Server

Run the server using nodemon:

nodemon server.js

The backend server will start at:

http://localhost:8000

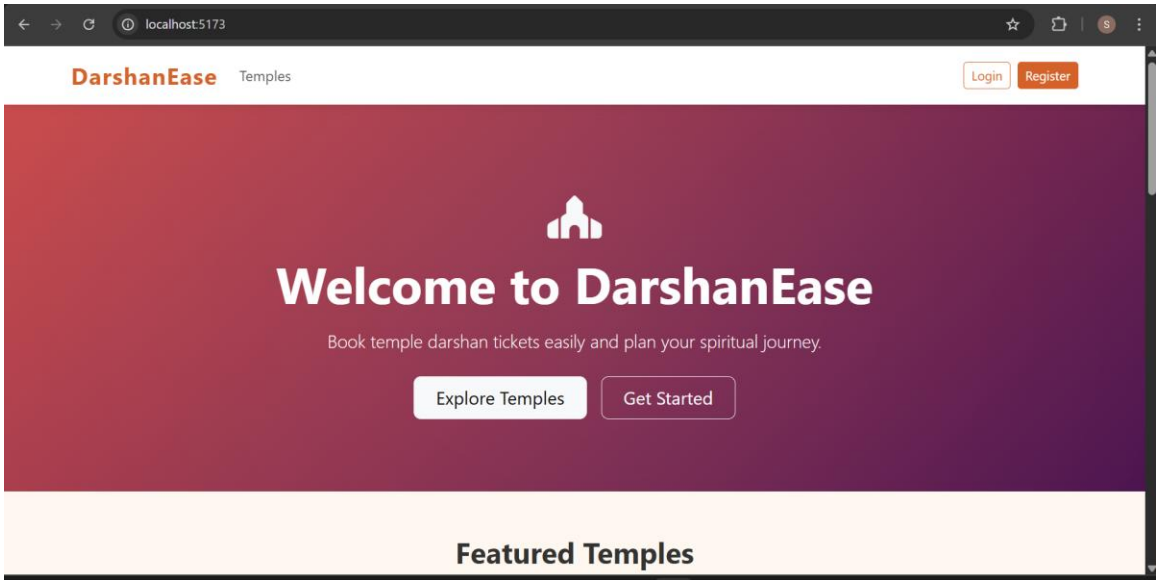
Final Output

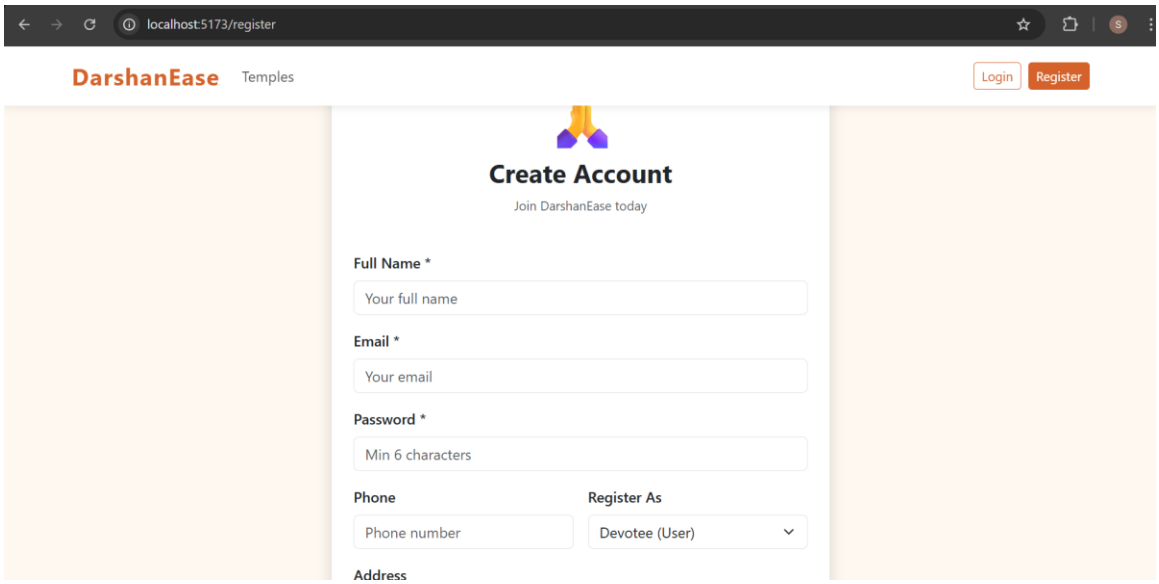
After starting both frontend and backend servers:

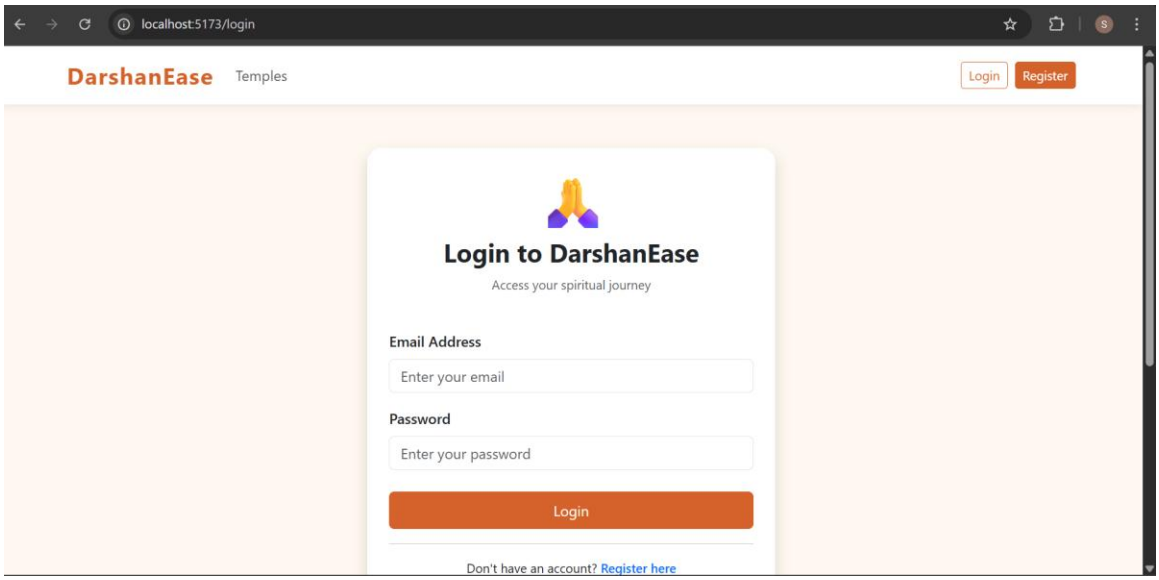
- Frontend: **http://localhost:5173**
- Backend: **http://localhost:8000**

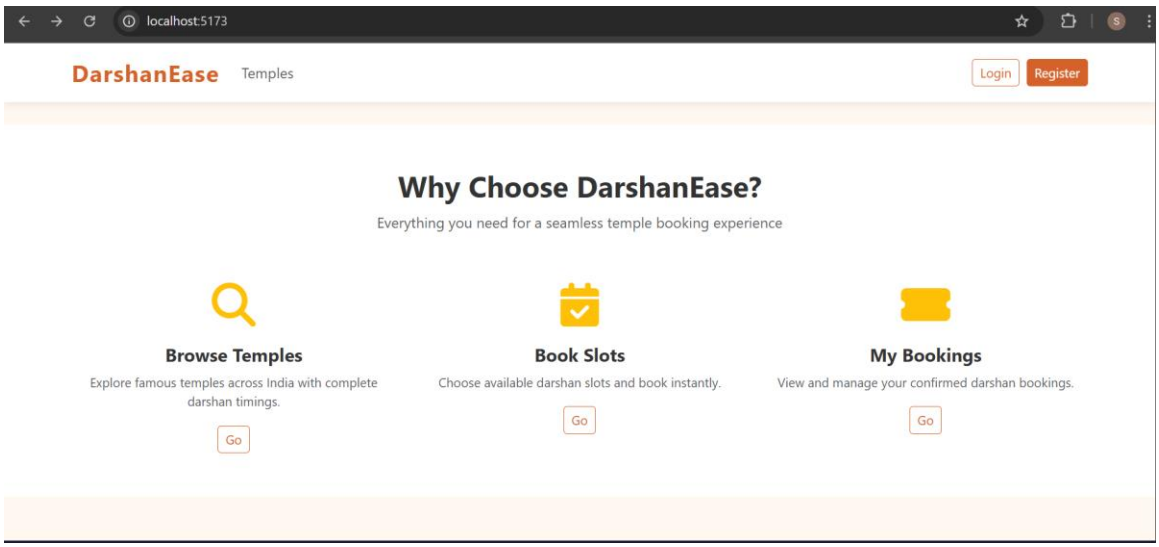
The DarshanEase system will now be fully functional and ready for use.

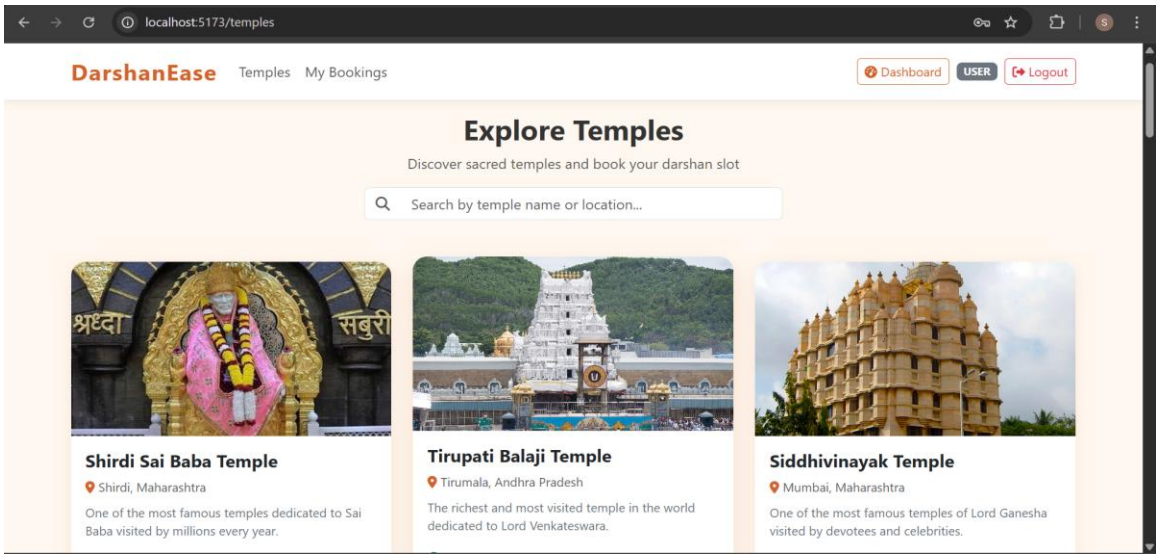
OUTPUT SCREENSHOTS:

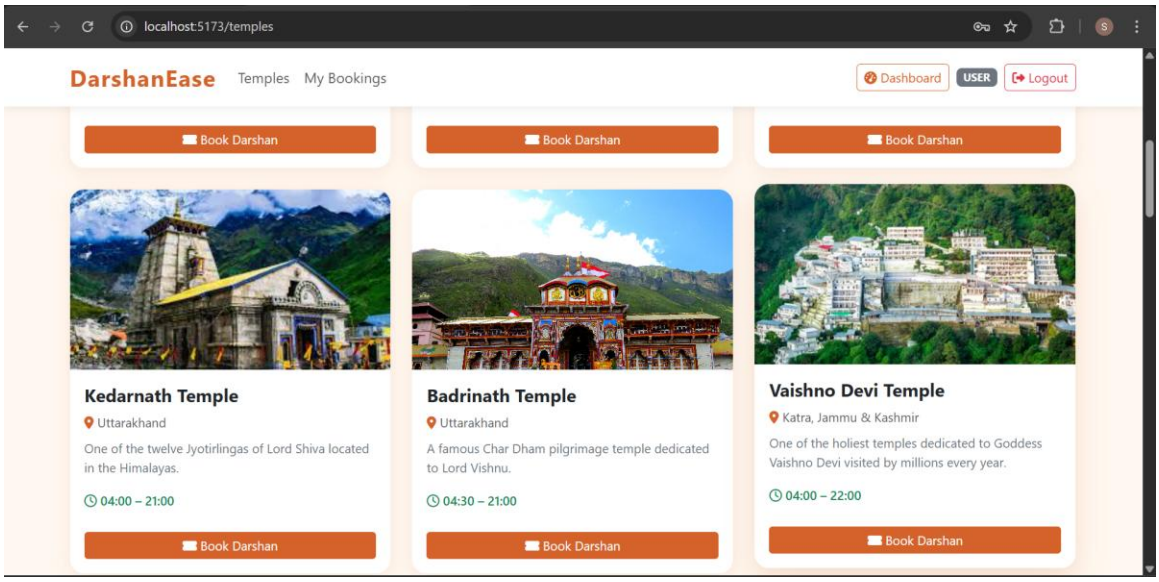


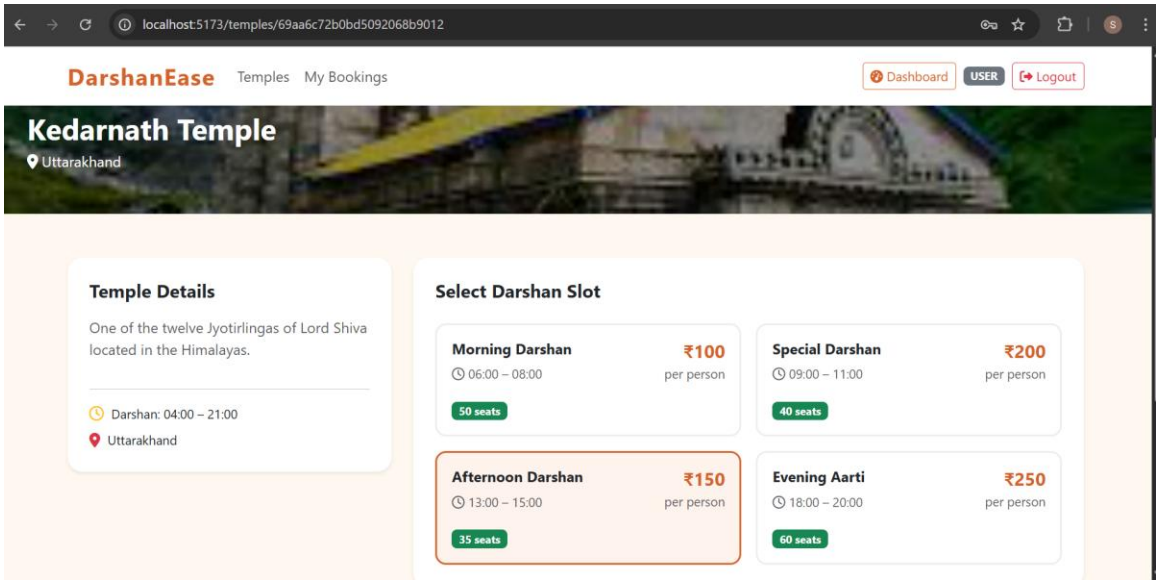


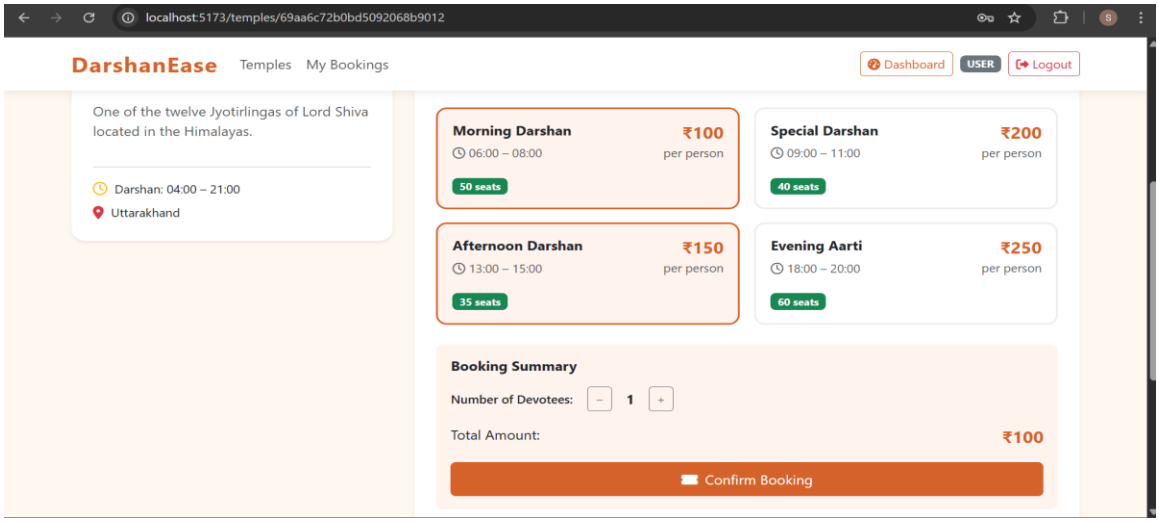

localhost:5173/login
DarshanEase
Temples
Login
Register
Login to DarshanEase
Access your spiritual journey
Email Address
Enter your email
Password
Enter your password
Login
Don't have an account? Register here

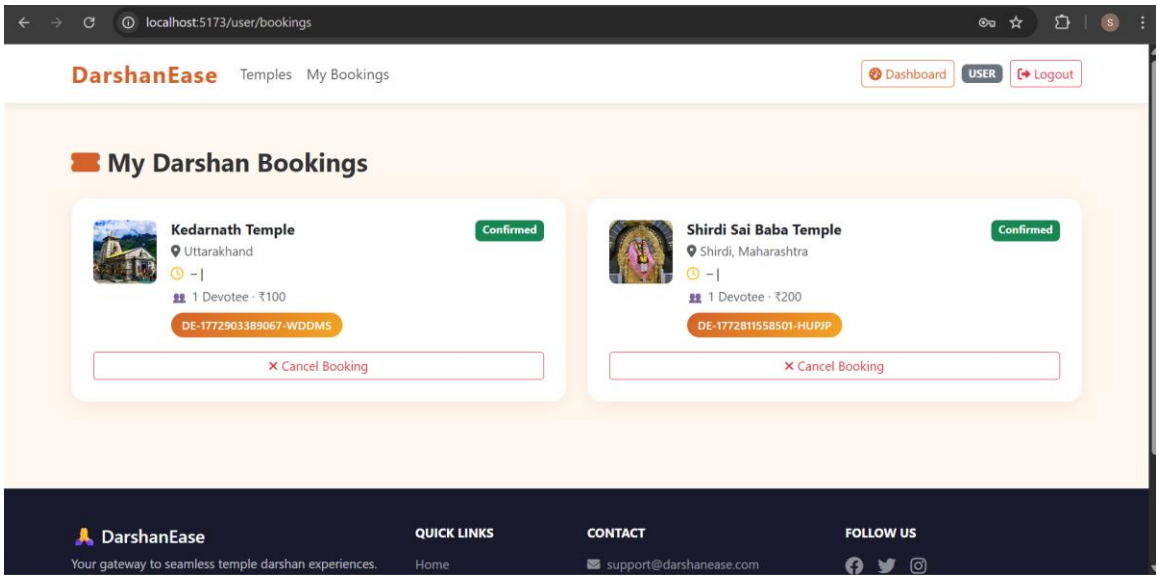

localhost:5173
DarshanEase
Temples
Login
Register
Why Choose DarshanEase?
Everything you need for a seamless temple booking experience
Browse Temples
Explore famous temples across India with complete darshan timings.
Go
Book Slots
Choose available darshan slots and book instantly.
Go
My Bookings
View and manage your confirmed darshan bookings.
Go


localhost:5173/temples
DarshanEase
Temples
My Bookings
Dashboard
USER
Logout
Explore Temples
Discover sacred temples and book your darshan slot
Search by temple name or location...
Shirdi Sai Baba Temple
Shirdi, Maharashtra
One of the most famous temples dedicated to Sai Baba visited by millions every year.
Tirupati Balaji Temple
Tirumala, Andhra Pradesh
The richest and most visited temple in the world dedicated to Lord Venkateswara.
Siddhivinayak Temple
Mumbai, Maharashtra
One of the most famous temples of Lord Ganesha visited by devotees and celebrities.


localhost:5173/temples
DarshanEase
Temples
My Bookings
Dashboard
USER
Logout
Book Darshan
Book Darshan
Book Darshan
Kedarnath Temple
Uttarakhand
One of the twelve Jyotirlingas of Lord Shiva located in the Himalayas.
04:00 – 21:00
Book Darshan
Badrinath Temple
Uttarakhand
A famous Char Dham pilgrimage temple dedicated to Lord Vishnu.
04:30 – 21:00
Book Darshan
Vaishno Devi Temple
Katra, Jammu & Kashmir
One of the holiest temples dedicated to Goddess Vaishno Devi visited by millions every year.
04:00 – 22:00
Book Darshan


localhost:5173/temples/69aa6c72b0bd5092068b9012
DarshanEase
Temples
My Bookings
Dashboard
USER
Logout
Kedarnath Temple
Uttarakhand
Temple Details
One of the twelve Jyotirlingas of Lord Shiva located in the Himalayas.
Darshan: 04:00 – 21:00
Uttarakhand
Select Darshan Slot
Morning Darshan
06:00 – 08:00
₹100
per person
50 seats
Special Darshan
09:00 – 11:00
₹200
per person
40 seats
Afternoon Darshan
13:00 – 15:00
₹150
per person
35 seats
Evening Aarti
18:00 – 20:00
₹250
per person
60 seats


localhost:5173/temples/69aa6c72b0bd5092068b9012
DarshanEase
Temples
My Bookings
Dashboard
USER
Logout
One of the twelve Jyotirlingas of Lord Shiva located in the Himalayas.
Darshan: 04:00 – 21:00
Uttarakhand
Morning Darshan
06:00 – 08:00
₹100
per person
50 seats
Special Darshan
09:00 – 11:00
₹200
per person
40 seats
Afternoon Darshan
13:00 – 15:00
₹150
per person
35 seats
Evening Aarti
18:00 – 20:00
₹250
per person
60 seats
Booking Summary
Number of Devotees:
1
Total Amount:
₹100
Confirm Booking


localhost:5173/user/bookings
DarshanEase
Temples
My Bookings
Dashboard
USER
Logout
My Darshan Bookings
Kedarnath Temple
Confirmed
Uttarakhand
– |
1 Devotee · ₹100
DE-1772903389067-WDDMS
✕ Cancel Booking
Shirdi Sai Baba Temple
Confirmed
Shirdi, Maharashtra
– |
1 Devotee · ₹200
DE-1772811558501-HUPJP
✕ Cancel Booking
DarshanEase
Your gateway to seamless temple darshan experiences.
QUICK LINKS
Home
CONTACT
support@darshanease.com
FOLLOW US